



47. T-Distributed Stochastic Neighbor Embedding

Seeing the Hidden Structure with t-SNE

Previous Section:

 [46. Active Learning](#)

Contents:

- [1. The t-SNE Algorithm - The Overview](#)
- [2. The t-SNE Algorithm - Iris Dataset](#)
- [3. The t-SNE Algorithm - Manual Calculations](#)
- [4. t-SNE with Python](#)

[Summary](#)

[References](#)

Let me get this straight first. That name is long. Really long. So we usually just call it t-SNE, because no one wants to say all of that every time. Now, what is it actually?

At its core, t-SNE is another dimensionality reduction method. It takes high-dimensional data and tries to compress it into something we can actually visualize, usually 2D or 3D. But unlike simpler methods, it does not just compress information blindly. It tries to preserve the structure of the data, especially how points relate to their neighbors.

Let me break it down clearly:

- We are dealing with high-dimensional data that is hard to visualize directly

- We want to map it into lower dimensions without losing important structure
- We care more about preserving local relationships than global geometry

Now here is the important question. If we already have PCA, why do we even need this?

Because PCA and t-SNE are not trying to solve the problem in the same way. PCA focuses on preserving variance using linear projections. t-SNE, on the other hand, focuses on preserving similarity between points, especially local clusters. That difference is exactly why t-SNE exists in the first place.

So before we go into the algorithm itself, we need to understand what makes it fundamentally different from PCA, and why that difference matters when we start visualizing complex data.

1. The t-SNE Algorithm - The Overview

Let me break this down in a clean way so it actually makes sense.

T-Distributed Stochastic Neighbor Embedding, or t-SNE, is a non-linear dimensionality reduction technique. We use it when we want to visualize high-dimensional data in a lower-dimensional space, usually 2D or 3D.

Now here is the key idea: t-SNE is not trying to preserve global structure. It is trying to preserve local structure. In other words, it cares about who is close to whom. That already makes it very different from PCA.

Let's compare it directly with PCA so the difference is clear:

- PCA is a linear method. It tries to project data onto directions of maximum variance. t-SNE is non-linear. It focuses on preserving similarity between nearby points
- PCA keeps the overall structure, but can miss small clusters. t-SNE can reveal hidden clusters, even if the global structure gets distorted

So you can think of PCA as a method that shows the "big picture", while t-SNE is more like a microscope that zooms into local neighborhoods.

Let's review about Dimensionality Reduction: It is the process of simplifying complex datasets. It does this by combining or compressing correlated features. The goal is to make data easier to analyze and faster to compute.

t-SNE takes this idea further, but in a very different way from PCA. Instead of projecting data using linear algebra, t-SNE starts by measuring similarity

between points in the high-dimensional space. It then converts those similarities into probabilities, meaning it asks how likely one point is to be a neighbor of another. Points that are similar get high probability. Points that are different get low probability. Then it tries to recreate those same relationships in a lower-dimensional space.

So in simple terms: t-SNE is not just reducing dimensions. It is rebuilding the data layout so that similar points stay close together, even after compression.

Now let me walk you through how t-SNE actually works, but in a way that connects directly to a real dataset instead of abstract theory.

2. The t-SNE Algorithm - Iris Dataset

Now let me walk you through how t-SNE actually works, but in a way that connects directly to a real dataset instead of abstract theory.

We'll use a simple example: the Iris dataset (without target labels because we are not predicting anything).

Sepal Length	Sepal Width	Petal Length	Petal Width
5.1	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.2
7.0	3.2	4.7	1.4
6.4	3.2	4.5	1.5
6.9	3.1	4.9	1.5
5.5	2.3	4.0	1.3
6.5	2.8	4.6	1.5
6.3	3.3	6.0	2.5
5.8	2.7	5.1	1.9
7.1	3.0	5.9	2.1
6.3	2.9	5.6	1.8
6.5	3.0	5.8	2.2

Now imagine we want to visualize this 4D data in 2D. That is exactly what t-SNE is designed for.

Step 1: Prepare the Data

First, t-SNE cleans the data:

- Remove rows with missing values (NaN)
- Optionally standardize each feature:
 - subtract mean
 - divide by standard deviation

So every feature is on the same scale.

Before going further, many implementations also do PCA first (like reducing to 50 dimensions). This is just to remove noise and speed things up.

So we can think of it like: **Raw data** → **cleaned data** → **(optional PCA)** → **ready for t-SNE**

Step 2: Compute Pairwise Distances

Now t-SNE looks at every pair of points. For two points x_i and x_j , we compute distance:

$$d(x_i, x_j) = \|x_i - x_j\|^2$$

So now we know how far every flower is from every other flower.

Step 3: Convert Distances into Probabilities

Now comes the key idea. Instead of just saying "close" or "far", t-SNE turns distance into probability:

$$p_{j|i} = \frac{\exp\left(-\frac{d(x_i, x_j)}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{d(x_i, x_k)}{2\sigma_i^2}\right)}$$

What this means in simple terms:

- If two points are close → probability is high

- If they are far → probability is almost zero

So instead of geometry, we now have a “likelihood of being neighbors”.

Step 4: Symmetrize the Probabilities

We don't want direction bias, so we combine both sides:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

Now we have a balanced similarity map of the dataset.

Step 5: Match Low-Dimensional Space

Now we create a new space (usually 2D). Each point starts randomly:

- We place each flower randomly on a 2D plane
- Then we define similarity again, but this time using a Student-t distribution:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|y_k - y_l\|^2)^{-1}}$$

So now we have:

- P : similarity in original space
- Q : similarity in 2D space

Step 6: Measure Mismatch (KL Divergence)

Now we compare them:

$$KL(P \parallel Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

This tells us: If P and Q match → small value; If they differ → large value

So the goal is simple: Make Q look like P

Step 7: Optimize (Gradient Descent)

Now we adjust the 2D points:

$$\frac{\partial KL}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) q_{ij} (y_i - y_j)$$

This is repeated many times:

- If points should be closer → they move closer
- If they are too close → they push apart

Extra tricks are used:

- Early exaggeration: clusters are pushed apart first to form clearer groups
- Learning rate: controls how fast points move

Let me simplify everything:

- Step 1: Measure real similarities in high dimensions
- Step 2: Convert them into probabilities
- Step 3: Randomly place points in 2D
- Step 4: Measure similarities again in 2D
- Step 5: Move points until both match

t-SNE is not trying to preserve shape. It is trying to preserve *relationships*. Not "what does this data look like?" → But rather: "Who is similar to whom, and can I make that visible?"

That is why t-SNE can reveal clusters that PCA completely hides. Because PCA draws a straight line through data. t-SNE rebuilds the neighborhood structure from scratch.

3. The t-SNE Algorithm - Manual Calculations

Let's be honest here. Computing t-SNE by hand is not just hard, it is basically a torture test for your patience. So instead of doing everything manually, we let NumPy handle the heavy lifting. Our job is not to avoid the math, but to correctly translate the formula into code.

First, we load the dataset as a NumPy array so we can work with it efficiently.

Here is how we start:

```
import numpy as np

# Iris dataset (ignoring target labels)
X = np.array([[5.1, 3.5, 1.4, 0.2],
              [4.9, 3.0, 1.4, 0.2],
              [4.7, 3.2, 1.3, 0.2],
              [4.6, 3.1, 1.5, 0.2],
              [5.0, 3.6, 1.4, 0.2],
              [7.0, 3.2, 4.7, 1.4],
              [6.4, 3.2, 4.5, 1.5],
              [6.9, 3.1, 4.9, 1.5],
              [5.5, 2.3, 4.0, 1.3],
              [6.5, 2.8, 4.6, 1.5],
              [6.3, 3.3, 6.0, 2.5],
              [5.8, 2.7, 5.1, 1.9],
              [7.1, 3.0, 5.9, 2.1],
              [6.3, 2.9, 5.6, 1.8],
              [6.5, 3.0, 5.8, 2.2]])
```

Now that we have the data, we begin applying the t-SNE logic step by step.

Step 1: Standardize the Data

t-SNE expects features to be on similar scales, so we normalize:

```
# Normalize the data
X = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
```

Step 2: Compute Pairwise Distances

We compute squared Euclidean distances between all points:

```
def compute_distances(X):
    sum_X = np.sum(np.square(X), axis=1)
    distances = sum_X[:, None] + sum_X[None, :] - 2 * np.dot(
        X, X.T)
```

```

    return distances

D = compute_distances(X)

```

At this point, we now know how far every flower is from every other flower.

Step 3: Convert Distances into Probabilities

Now we apply the Gaussian similarity idea:

```

def compute_pij(D, sigma=1.0):
    P = np.exp(-D / (2 * sigma**2))
    np.fill_diagonal(P, 0)
    P = P / np.sum(P)
    return P

P = compute_pij(D)

```

This gives us a probability matrix where:

- Close points → high probability
- Far points → near zero

So instead of distance, we now have “how likely are they neighbors”.

Step 4: Initialize Low-Dimensional Embedding

We randomly place points in 2D space:

```

n = X.shape[0]
Y = np.random.randn(n, 2)

```

Step 5: Compute Low-Dimensional Similarities

We now compute similarities in 2D using the Student-t distribution:

```

def compute_qij(Y):
    sum_Y = np.sum(np.square(Y), axis=1)

```

```

    num = 1 / (1 + sum_Y[:, None] + sum_Y[None, :] - 2 * n
p.dot(Y, Y.T))
    np.fill_diagonal(num, 0)
    Q = num / np.sum(num)
    return Q

```

Now we have:

- P → original structure
- Q → current 2D structure

Step 6: Measure Mismatch (KL Divergence)

We compare both distributions:

```

def kl_divergence(P, Q):
    return np.sum(P * np.log((P + 1e-10) / (Q + 1e-10)))

```

- If KL is high → structure is wrong
- If KL is low → structure matches reality

Step 7: Gradient Update (Learning Step)

Now we move the points:

```

def update(Y, P, Q, lr=0.1):
    n = Y.shape[0]
    for i in range(n):
        grad = np.zeros(2)
        for j in range(n):
            if i != j:
                diff = (P[i, j] - Q[i, j]) * (Y[i] - Y[j])
                grad += diff
        Y[i] += lr * grad
    return Y

```

This step is repeated many times:

- If two points should be closer → they move closer
- If they are too close → they push apart

Final Loop (Putting Everything Together)

```
for epoch in range(100):
    Q = compute_qij(Y)
    loss = kl_divergence(P, Q)
    Y = update(Y, P, Q)

    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss}")
```

(Here is the full code)

[t-sne.py](#)

What we just built is not magic. It is simply this:

- Measure relationships in high dimensions
- Try to recreate them in 2D
- Keep adjusting until both worlds match

But the important part is not the code. It is the idea behind it: t-SNE does not compress data. It reconstructs the neighborhood structure in a simpler space. And that is why, when it works, it feels almost like the data is revealing its hidden map.

4. t-SNE with Python

We can implement the t-SNE algorithm directly using Scikit-Learn in Python:

```

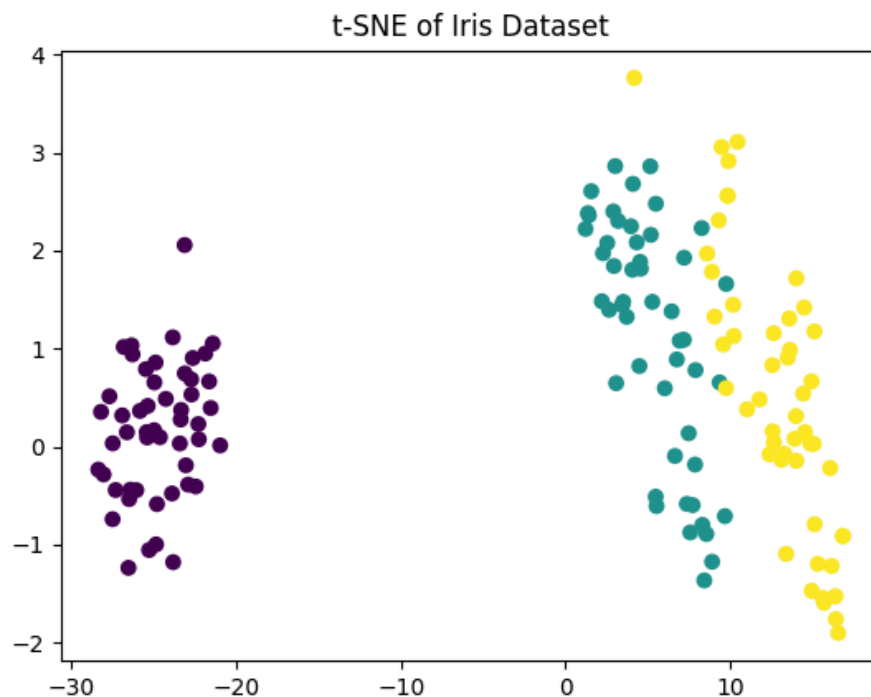
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Load Data
iris = load_iris()

# Initialize and Fit t-SNE
# n_components=2 reduces the 4D data to 2D
tsne = TSNE(n_components=2, random_state=42)
X_embedded = tsne.fit_transform(iris.data)

# Visualize
plt.scatter(X_embedded[:, 0], X_embedded[:, 1], c=iris.target)
plt.title("t-SNE of Iris Dataset")
plt.show()

```



Summary

t-SNE (t-distributed Stochastic Neighbor Embedding) is that flashy alternative to PCA everyone reaches for when they want their high-dimensional data to *look* cool in 2D or 3D.

Instead of just projecting data along directions of maximum variance like PCA does, t-SNE focuses on preserving local structure—basically, it tries really hard to keep similar points close together in the lower-dimensional space. It does this by turning distances into probabilities and then minimizing how different those probability distributions are between the high-dimensional and low-dimensional versions.

The result? Clusters that often look super clean and separated, even when the original data is messy.

But t-SNE isn't magic. It's great for visualization, not so great for preserving global structure—so distances between clusters don't always mean much. It's also sensitive to parameters like perplexity, can be slow on large datasets, and isn't deterministic unless you lock the random seed.

In short, PCA is like a reliable, straightforward tool, while t-SNE is more like an artistic filter: amazing for exploring patterns and spotting clusters, but you shouldn't over-interpret the picture it paints.

References

<https://www.geeksforgeeks.org/machine-learning/ml-t-distributed-stochastic-neighbor-embedding-t-sne-algorithm/>

<https://www.ultralytics.com/glossary/t-distributed-stochastic-neighbor-embedding-t-sne>

<https://www.mathworks.com/help/stats/tsne.html>

Next Section:

 **48. Neural Networks**